

LAPORAN PRAKTIKUM

Algoritma & Struktur Data

Linked List



DISUSUN OLEH:

FAQROEL RAZI

2025573010033

TEKNIK INFORMATIKA

POLITEKNIK NEGERI LHOKSEUMAWE

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum yang berjudul "**Linked List**" dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum mata kuliah Struktur Data yang membahas implementasi serta penerapan konsep Linked List dalam pemrograman.

Dalam penyusunan laporan ini, penulis memperoleh banyak pengetahuan dan pemahaman mengenai struktur data Linked List, mulai dari konsep dasar Node dan Linked List, proses penggabungan data (merge), pendeteksian siklus (cycle), hingga penerapan algoritma Floyd's Cycle Detection. Materi tersebut memberikan pemahaman yang lebih mendalam mengenai pengelolaan data secara dinamis dalam pengembangan perangkat lunak.

Penulis mengucapkan terima kasih kepada **Bapak M. Reza Zulman, M.Sc.** selaku dosen pengampu yang telah memberikan arahan, materi, dan bimbingan selama kegiatan praktikum berlangsung. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membantu dalam penyusunan laporan ini.

Penulis menyadari bahwa laporan ini masih memiliki berbagai kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca serta menjadi referensi dalam mempelajari struktur data Linked List.

Lhokseumawe, 5 Juni 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM.....	i
KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Identifikasi Masalah	1
1.3 Rumusan Masalah	2
BAB II LANDASAN TEORI	3
2.1 Linked List	3
2.2 Class Node dan Linked List Dasar.....	3
2.3 Siklus dan Merge pada Linked List	4
2.4 Floyd's Cycle Detection	5
BAB III METODOLOGI PRAKTIKUM.....	7
3.1 Alat dan Bahan	7
3.2 Prosedur Praktikum.....	7
3.2.1 Setup project.....	7
3.2.2 Melakukan tugas-1 Doubly Linked List	8
3.2.3 Melakukan tugas-2 Soal Klasik Linked List.....	12
3.2.4 Melakukan integrasi data antara komputer lokal dengan repositori GitHub.....	16
BAB IV HASIL DAN PEMBAHASAN.....	17
4.1 Hasil Praktikum.....	17

4.2 Pembahasan.....	17
BAB V PENUTUP.....	18
5.1. Kesimpulan	18
5.2. Saran.....	18
DAFTAR PUSTAKA.....	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam ilmu komputer yang digunakan untuk mengatur, menyimpan, dan mengelola data secara efisien. Pemilihan struktur data yang tepat dapat memengaruhi performa suatu program, terutama ketika menangani data dalam jumlah besar. Oleh karena itu, pemahaman mengenai berbagai jenis struktur data menjadi hal yang penting bagi mahasiswa bidang teknologi informasi.

Salah satu struktur data yang banyak digunakan adalah Linked List. Linked List merupakan struktur data linear yang tersusun atas kumpulan node yang saling terhubung melalui pointer atau referensi. Berbeda dengan array yang menggunakan memori berurutan, Linked List memungkinkan alokasi memori secara dinamis sehingga lebih fleksibel dalam proses penambahan maupun penghapusan data.

Pada praktikum ini dilakukan pembelajaran mengenai implementasi Linked List beserta berbagai operasinya. Selain mempelajari konsep dasar node dan Linked List, praktikum juga membahas teknik penggabungan Linked List, deteksi siklus pada Linked List, serta penerapan algoritma Floyd's Cycle Detection. Materi tersebut bertujuan untuk meningkatkan pemahaman mahasiswa mengenai struktur data dinamis dan penerapannya dalam pemrograman.

1.2 Identifikasi Masalah

Berdasarkan latar belakang tersebut, dapat diidentifikasi beberapa permasalahan sebagai berikut:

1. Bagaimana konsep dasar struktur data Linked List.
2. Bagaimana implementasi Node dan Linked List dalam pemrograman.
3. Bagaimana proses penggabungan (merge) pada Linked List.

4. Bagaimana mendeteksi adanya siklus (cycle) pada Linked List.
5. Bagaimana cara kerja algoritma Floyd's Cycle Detection.

1.3 Rumusan Masalah

Adapun rumusan masalah pada praktikum ini adalah sebagai berikut:

1. Apa yang dimaksud dengan Linked List dan bagaimana karakteristiknya?
2. Bagaimana hubungan antara Node dan Linked List dalam pembentukan struktur data?
3. Bagaimana mekanisme penggabungan dua Linked List?
4. Apa yang dimaksud dengan siklus pada Linked List?
5. Bagaimana algoritma Floyd's Cycle Detection digunakan untuk mendeteksi siklus?

BAB II

LANDASAN TEORI

2.1 Linked List

Linked List adalah struktur data linear yang terdiri dari sejumlah node yang saling terhubung. Setiap node menyimpan data dan referensi menuju node berikutnya. Berbeda dengan array yang menggunakan lokasi memori berurutan, Linked List menggunakan alokasi memori dinamis sehingga elemen dapat ditambahkan atau dihapus tanpa harus menggeser seluruh data.

Keunggulan Linked List antara lain fleksibilitas penggunaan memori, kemudahan penambahan dan penghapusan data, serta kemampuan menangani ukuran data yang berubah-ubah. Namun, Linked List memiliki kelemahan berupa akses data yang harus dilakukan secara berurutan sehingga pencarian data memerlukan waktu lebih lama dibandingkan array.

Jenis-jenis Linked List meliputi:

Singly Linked List

Doubly Linked List

Circular Linked List

2.2 Class Node dan Linked List Dasar

Node merupakan komponen utama penyusun Linked List. Sebuah node umumnya terdiri atas dua bagian yaitu:

Data, berisi nilai yang disimpan.

Next, berisi referensi ke node berikutnya.

Contoh struktur Node:

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

Linked List dibangun dengan menghubungkan beberapa node melalui atribut next. Node pertama disebut head yang menjadi titik awal akses seluruh data pada Linked List.

Contoh struktur Linked List:

```
class LinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

Operasi dasar pada Linked List meliputi:

Insertion (penambahan data)

Deletion (penghapusan data)

Traversal (penelusuran data)

Searching (pencarian data)

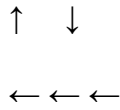
2.3 Siklus dan Merge pada Linked List

Siklus (Cycle)

Siklus pada Linked List terjadi ketika suatu node kembali menunjuk ke node sebelumnya sehingga membentuk perulangan. Akibatnya proses traversal dapat berjalan tanpa akhir apabila tidak dilakukan pemeriksaan.

Contoh:

1 → 2 → 3 → 4



Siklus sering muncul akibat kesalahan manipulasi pointer dalam program.

Merge Linked List

Merge adalah proses menggabungkan dua Linked List menjadi satu Linked List baru. Teknik ini sering digunakan pada algoritma pengurutan seperti Merge Sort.

Contoh:

List A : $1 \rightarrow 3 \rightarrow 5$

List B : $2 \rightarrow 4 \rightarrow 6$

Hasil :

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$

Tujuan utama proses merge adalah mengombinasikan data secara terstruktur dan efisien.

2.4 Floyd's Cycle Detection

Floyd's Cycle Detection Algorithm atau dikenal sebagai Tortoise and Hare Algorithm merupakan metode yang digunakan untuk mendeteksi siklus pada Linked List.

Algoritma ini menggunakan dua pointer:

Slow Pointer (bergerak satu langkah).

Fast Pointer (bergerak dua langkah).

Prinsip kerjanya adalah:

Jika tidak terdapat siklus, fast pointer akan mencapai nilai NULL terlebih dahulu.

Jika terdapat siklus, fast pointer dan slow pointer akan bertemu pada titik tertentu di dalam siklus.

Keunggulan algoritma ini:

Kompleksitas waktu $O(n)$.

Kompleksitas ruang $O(1)$.

Tidak membutuhkan struktur data tambahan.

Algoritma ini menjadi salah satu metode paling efisien dalam mendeteksi cycle pada Linked List.

BAB III

METODOLOGI PRAKTIKUM

3.1 Alat dan Bahan

Berdasarkan praktikum yang dilakukan, alat dan bahan yang diperlukan adalah:

- Komputer atau Laptop
- IDE Visual Studio Code
- Node JS
- Git
- GitHub

3.2 Prosedur Praktikum

3.2.1 Setup project

Membuat folder tugas untuk menyimpan project website todo list:

1. Buka git bash dan navigasikan ke folder repository GitHub yang telah di clone.
2. Buat direktori praktikum-6
3. Masuk ke direktori praktikum-6 lalu
4. Buat project node didalam direktori
5. Buat direktori tugas
6. Masuk ke direktori tugas lalu
7. Buat file tugas-1.js dan tugas-2.js

```

ASUSQLAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma (main)
• $ mkdir praktikum_6

ASUSQLAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma (main)
• $ cd praktikum_6/

ASUSQLAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_6 (main)
• $ npm init -y
Wrote to D:\2025573010033_Prak-Struktur-Data-dan-Algoritma\praktikum_6\package.json:

{
  "name": "praktikum_6",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

ASUSQLAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_6 (main)
• $ mkdir tugas

ASUSQLAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_6 (main)
• $ cd tugas/

ASUSQLAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_6/tugas (main)
• $ touch tugas-1.js tugas-2.js

ASUSQLAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_6/tugas (main)
o $ █

```

3.2.2 Melakukan tugas-1 Doubly Linked List

Buka file tugas-1.js yang telah dibuat dan buatlah code javascript untuk memenuhi permintaan tugas pada modul praktikum:

Source code tugas-1.js:

```
tugas-1.js x
praktikum-6 > tugas > tugas-1.js > DoublyLinkedList

1  class Node {
    Windsurf: Refactor | Explain | Generate JSDoc
2  ...constructor(data) {
3  ...  this.data = data;
4  ...  this.next = null;
5  ...  this.prev = null;
6  ...}
7  }
8
9  class DoublyLinkedList {
    Windsurf: Refactor | Explain | Generate JSDoc
10 ...constructor() {
11 ...  this.head = null;
12 ...  this.tail = null; // O(1) access to last node
13 ...  this.size = 0;
14 ...}
15
16 ...// append: O(1)
    Windsurf: Refactor | Explain
17 ...append(data) {
18 ...  const node = new Node(data);
19 ...  if (!this.head) {
20 ...    this.head = node;
21 ...    this.tail = node;
22 ...  } else {
23 ...    this.tail.next = node;
24 ...    node.prev = this.tail;
25 ...    this.tail = node;
26 ...  }
27 ...  this.size++;
28 ...}
29
30 ...// prepend: O(1)
    Windsurf: Refactor | Explain
31 ...prepend(data) {
32 ...  const node = new Node(data);
33 ...  if (!this.head) {
34 ...    this.head = node;
35 ...    this.tail = node;
36 ...  } else {
37 ...    node.next = this.head;
38 ...    this.head.prev = node;
39 ...    this.head = node;
40 ...  }
41 ...  this.size++;
42 ...}
```

```

tugas-1.js X
praktikum-6 > tugas > tugas-1.js > DoublyLinkedList
44 // insertAt: O(n): {}
Windsurf: Refactor | Explain
45 insertAt(data, index) {
46   if (index < 0 || index > this.size) {
47     console.log("Index di luar batas!");
48     return false;
49   }
50   if (index === 0) {
51     this.prepend(data);
52     return true;
53   }
54   if (index === this.size) {
55     this.append(data);
56     return true;
57   }
58
59   const node = new Node(data);
60   // choose traversal direction for efficiency
61   let current;
62   if (index <= this.size / 2) {
63     current = this.head;
64     for (let i = 0; i < index - 1; i++) current = current.next;
65   } else {
66     current = this.tail;
67     for (let i = this.size - 1; i > index - 1; i--) current = current.prev;
68   }
69
70   node.next = current.next;
71   node.prev = current;
72   current.next.prev = node;
73   current.next = node;
74   this.size++;
75   return true;
76 }
77

```

```

tugas-1.js X
praktikum-6 > tugas > tugas-1.js > DoublyLinkedList
78 // delete by value: O(n)
Windsurf: Refactor | Explain
79 delete(data) {
80   if (!this.head) return false;
81   let current = this.head;
82   while (current) {
83     if (current.data === data) {
84       if (current.prev) current.prev.next = current.next;
85       else this.head = current.next; // removing head
86
87       if (current.next) current.next.prev = current.prev;
88       else this.tail = current.prev; // removing tail
89
90       this.size--;
91       return true;
92     }
93     current = current.next;
94   }
95   return false;
96 }
97
98 // reverse list in-place: O(n)
Windsurf: Refactor | Explain
99 reverse() {
100   let current = this.head;
101   let temp = null;
102   while (current) {
103     // swap next and prev
104     temp = current.prev;
105     current.prev = current.next;
106     current.next = temp;
107     current = current.prev; // we swapped, so move to previous
108   }
109   // swap head and tail
110   temp = this.head;
111   this.head = this.tail;
112   this.tail = temp;
113 }
114

```

```

tugas-1.js x
praktikum-6 > tugas > tugas-1.js > DoublyLinkedList
115 // print from head: O(n)
    Windsurf: Refactor | Explain
116 printForward() {
117     const values = [];
118     let current = this.head;
119     while (current) {
120         values.push(current.data);
121         current = current.next;
122     }
123     console.log("Forward:", values.join("<->"), `(size: ${this.size})`);
124 }
125
126 // print from tail: O(n)
    Windsurf: Refactor | Explain
127 printBackward() {
128     const values = [];
129     let current = this.tail;
130     while (current) {
131         values.push(current.data);
132         current = current.prev;
133     }
134     console.log("Backward:", values.join("<->"), `(size: ${this.size})`);
135 }
136
137 // helper: isEmpty: O(1)
    Windsurf: Refactor | Explain
138 isEmpty() {
139     return this.size === 0;
140 }
141
142
143 // --- Contoh penggunaan / pengujian singkat ---
144 if (require && require.main === module) {
145     const dll = new DoublyLinkedList();
146     console.log("Mulai pengujian DoublyLinkedList");
147
148     // append O(1)
149     dll.append(10);
150     dll.append(20);
151     dll.append(30);
152     dll.printForward();
153     dll.printBackward();
154 }

```

```

tugas-1.js x
praktikum-6 > tugas > tugas-1.js > DoublyLinkedList
155 // prepend O(1)
156 dll.prepend(5);
157 dll.printForward();
158
159 // insertAt O(n)
160 dll.insertAt(15, 2); // antara 10 dan 20
161 dll.printForward();
162
163 // delete O(n)
164 dll.delete(20);
165 dll.printForward();
166 dll.printBackward();
167
168 // reverse O(n)
169 dll.reverse();
170 dll.printForward();
171 dll.printBackward();
172
173 // demonstrasi tail O(1) append
174 console.log("Append lagi (menunjukkan tail O(1))");
175 dll.append(100);
176 dll.printBackward();
177 }
178
179 module.exports = { DoublyLinkedList, Node };
180

```

Hasil yang didapatkan:

```
ASUSLAPTOP-F10506TJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum-6/tugas (main)
$ node tugas-1.js
Mulai pengujian DoublyLinkedList
Forward: 10 <-> 20 <-> 30 (size: 3)
Backward: 30 <-> 20 <-> 10 (size: 3)
Forward: 5 <-> 10 <-> 20 <-> 30 (size: 4)
Forward: 5 <-> 10 <-> 15 <-> 20 <-> 30 (size: 5)
Forward: 5 <-> 10 <-> 15 <-> 30 (size: 4)
Backward: 30 <-> 15 <-> 10 <-> 5 (size: 4)
Forward: 30 <-> 15 <-> 10 <-> 5 (size: 4)
Backward: 5 <-> 10 <-> 15 <-> 30 (size: 4)
Append lagi (menunjukkan tail 0(1))
Backward: 100 <-> 5 <-> 10 <-> 15 <-> 30 (size: 5)
$
```

3.2.3 Melakukan tugas-2 Soal Klasik Linked List

Buka file tugas-2.js yang telah dibuat dan buatlah code javascript untuk memenuhi permintaan tugas pada modul praktikum:

Source code tugas-2.js:

```
tugas-2.js X
praktikum-6 > tugas > tugas-2.js > hapusNDariAkhir
1  // Class Node untuk Linked List
   Windsurf: Refactor | Explain
2  class Node {
   Windsurf: Refactor | Explain | Generate JSDoc
3      constructor(value) {
4          this.value = value;
5          this.next = null;
6      }
7  }
8
9  // Cek apakah Linked List adalah palindrom
   Windsurf: Refactor | Explain
10 function palindromLL(head) {
11     if (!head) return true;
12
13     // Konversi ke array
14     const arr = [];
15     let current = head;
16     while (current) {
17         arr.push(current.value);
18         current = current.next;
19     }
20
21     // Cek apakah array adalah palindrom
22     let left = 0, right = arr.length - 1;
23     while (left < right) {
24         if (arr[left] !== arr[right]) return false;
25         left++;
26         right--;
27     }
28     return true;
29 }
30
31 // Hapus node ke-n dari akhir
   Windsurf: Refactor | Explain
32 function hapusNDariAkhir(head, n) {
33     // Buat dummy node untuk menangani kasus hapus head
34     const dummy = new Node(0);
35     dummy.next = head;
36
37     let first = dummy;
38     let second = dummy;
39 }
```

```
tugas-2.js X
praktikum-6 > tugas > tugas-2.js > hapusNDariAkhir

40  // Geser first pointer n+1 langkah
41  for (let i = 0; i <= n; i++) {
42    if (first.next === null) return head;
43    first = first.next;
44  }
45
46  // Geser both pointer hingga first mencapai akhir
47  while (first) {
48    first = first.next;
49    second = second.next;
50  }
51
52  // Hapus node
53  second.next = second.next.next;
54  return dummy.next;
55 }
56
57 // Kembalikan node tengah
58 function tengahLinkedList(head) {
59   if (!head) return null;
60
61   let slow = head;
62   let fast = head;
63
64   // Geser fast pointer 2 langkah, slow pointer 1 langkah
65   while (fast && fast.next) {
66     slow = slow.next;
67     fast = fast.next.next;
68   }
69
70   return slow;
71 }
72
73 // ===== TEST CASE =====
74 // Fungsi helper untuk membuat linked list dari array
75 function createLL(arr) {
76   if (arr.length === 0) return null;
77   const head = new Node(arr[0]);
78   let current = head;
79   for (let i = 1; i < arr.length; i++) {
80     current.next = new Node(arr[i]);
81     current = current.next;
82   }
83   return head;
```

```

praktikum-6 > tugas > tugas-2.js > hapusNDariAkhir
86 // Fungsi helper untuk print linked list
87 function printLL(head) {
88   let result = [];
89   let current = head;
90   while (current) {
91     result.push(current.value);
92     current = current.next;
93   }
94   return '[' + result.join(',') + ']';
95 }
96
97 console.log('==== TEST PALINDROM LL ====');
98
99 // Palindrom - [1, 2, 3, 2, 1]
100 const test1_1 = createLL([1, 2, 3, 2, 1]);
101 console.log('Test 1:', printLL(test1_1), '→', palindromLL(test1_1), '(expected: true)');
102
103 // Bukan palindrom - [1, 2, 3, 4, 5]
104 const test1_2 = createLL([1, 2, 3, 4, 5]);
105 console.log('Test 2:', printLL(test1_2), '→', palindromLL(test1_2), '(expected: false)');
106
107 // Single node - [5]
108 const test1_3 = createLL([5]);
109 console.log('Test 3:', printLL(test1_3), '→', palindromLL(test1_3), '(expected: true)');
110
111 console.log('\n==== TEST HAPUS N DARI AKHIR ====');
112
113 // Hapus node ke-2 dari akhir di [1, 2, 3, 4, 5]
114 const test2_1 = createLL([1, 2, 3, 4, 5]);
115 const result2_1 = hapusNDariAkhir(test2_1, 2);
116 console.log('Test 1: Hapus node ke-2 dari [1, 2, 3, 4, 5] →', printLL(result2_1), '(expected: [1, 2, 3, 4])');
117
118 // Hapus head (node ke-5 dari akhir) di [1, 2, 3, 4, 5]
119 const test2_2 = createLL([1, 2, 3, 4, 5]);
120 const result2_2 = hapusNDariAkhir(test2_2, 5);
121 console.log('Test 2: Hapus node ke-5 dari [1, 2, 3, 4, 5] →', printLL(result2_2), '(expected: [1, 2, 3, 4])');
122
123 // Hapus tail (node ke-1 dari akhir) di [1, 2, 3, 4, 5]
124 const test2_3 = createLL([1, 2, 3, 4, 5]);
125 const result2_3 = hapusNDariAkhir(test2_3, 1);
126 console.log('Test 3: Hapus node ke-1 dari [1, 2, 3, 4, 5] →', printLL(result2_3), '(expected: [1, 2, 3, 4])');
127
128 console.log('\n==== TEST TENGAH LINKED LIST ====');
129

```

```

praktikum-6 > tugas > tugas-2.js > hapusNDariAkhir
130 // List ganjil - [1, 2, 3, 4, 5] → node tengah = 3
131 const test3_1 = createLL([1, 2, 3, 4, 5]);
132 const result3_1 = tengahLinkedList(test3_1);
133 console.log('Test 1: Tengah dari [1, 2, 3, 4, 5] → value:', result3_1.value, '(expected: 3)');
134
135 // List genap - [1, 2, 3, 4] → node tengah-kedua = 3
136 const test3_2 = createLL([1, 2, 3, 4]);
137 const result3_2 = tengahLinkedList(test3_2);
138 console.log('Test 2: Tengah dari [1, 2, 3, 4] → value:', result3_2.value, '(expected: 3)');
139
140 // Single node - [10] → node tengah = 10
141 const test3_3 = createLL([10]);
142 const result3_3 = tengahLinkedList(test3_3);
143 console.log('Test 3: Tengah dari [10] → value:', result3_3.value, '(expected: 10)');
144
145

```

Hasil yang didapatkan:

```

ASUS@LAPTOP-FLOS6GTJ MINGW64 /d/2025573810033_Prak-Struktur-Data-dan-Algoritma/praktikum-6/tugas (main)
$ node tugas-2.js
==== TEST PALINDROM LL ====
Test 1: [1, 2, 3, 2, 1] → true (expected: true)
Test 2: [1, 2, 3, 4, 5] → false (expected: false)
Test 3: [5] → true (expected: true)

==== TEST HAPUS N DARI AKHIR ====
Test 1: Hapus node ke-2 dari [1, 2, 3, 4, 5] → [1, 2, 3, 4] (expected: [1, 2, 3, 4])
Test 2: Hapus node ke-5 dari [1, 2, 3, 4, 5] → [1, 2, 3, 4, 5] (expected: [2, 3, 4, 5])
Test 3: Hapus node ke-1 dari [1, 2, 3, 4, 5] → [1, 2, 3, 4] (expected: [1, 2, 3, 4])

==== TEST TENGAH LINKED LIST ====
Test 1: Tengah dari [1, 2, 3, 4, 5] → value: 3 (expected: 3)
Test 2: Tengah dari [1, 2, 3, 4] → value: 3 (expected: 3)
Test 3: Tengah dari [10] → value: 10 (expected: 10)
ASUS@LAPTOP-FLOS6GTJ MINGW64 /d/2025573810033_Prak-Struktur-Data-dan-Algoritma/praktikum-6/tugas (main)
$

```

3.2.4 Melakukan integrasi data antara komputer lokal dengan repositori GitHub

Lakukan integrasi proyek dari komputer lokal ke repositori GitHub dengan mengunggah seluruh berkas yang diperlukan sehingga setiap perubahan yang dilakukan dapat terdokumentasi dan tersimpan secara aman.

```
ASUS@LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_6/tugas (main)
• $ git add .

ASUS@LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_6/tugas (main)
• $ git commit -m "Praktikum 6: Linked List"
[main 9fb9785] Praktikum 6: Linked List
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 praktikum_6/tugas/tugas-1.js
 create mode 100644 praktikum_6/tugas/tugas-2.js

ASUS@LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_6/tugas (main)
• $ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 397 bytes | 99.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/FqrIRazi/2025573010033_Prak-Struktur-Data-dan-Algoritma.git
 c8440e3..9fb9785  main -> main

ASUS@LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_6/tugas (main)
○ $
```

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum

Hasil praktikum menunjukkan bahwa konsep Linked List dapat diimplementasikan dengan baik melalui penggunaan class Node dan Linked List. Program yang dikembangkan mampu melakukan penyimpanan data secara dinamis, menghubungkan node secara berurutan, menggabungkan beberapa Linked List, serta mendeteksi adanya siklus menggunakan algoritma Floyd's Cycle Detection secara efisien.

4.2 Pembahasan

Praktikum ini berhasil mengimplementasikan struktur data Linked List menggunakan bahasa pemrograman Python. Setiap node dapat menyimpan data dan referensi menuju node berikutnya sehingga membentuk rangkaian data yang saling terhubung. Melalui implementasi tersebut, mahasiswa dapat memahami bagaimana proses penambahan, penghapusan, dan penelusuran data dilakukan secara dinamis tanpa bergantung pada alokasi memori yang berurutan seperti pada array.

Selain implementasi dasar Linked List, praktikum juga membahas proses merge dan pendeteksian siklus menggunakan algoritma Floyd's Cycle Detection. Hasil pengujian menunjukkan bahwa algoritma tersebut mampu mendeteksi keberadaan siklus secara efisien dengan menggunakan dua pointer yang bergerak pada kecepatan berbeda. Pemahaman terhadap konsep ini sangat penting karena banyak digunakan dalam pengembangan perangkat lunak, terutama pada pengolahan struktur data dinamis dan optimasi algoritma.

BAB V

PENUTUP

5.1. Kesimpulan

Linked List merupakan struktur data dinamis yang tersusun atas kumpulan node yang saling terhubung. Setiap node berfungsi sebagai penyimpan data sekaligus memiliki referensi menuju node berikutnya sehingga membentuk suatu rangkaian data. Dalam implementasinya, proses merge digunakan untuk menggabungkan dua Linked List menjadi satu struktur data yang lebih besar dan terorganisir. Selain itu, siklus pada Linked List dapat menyebabkan proses traversal berjalan tanpa akhir apabila tidak dideteksi dengan baik. Untuk mengatasi permasalahan tersebut, Floyd's Cycle Detection dapat digunakan sebagai algoritma yang efisien dalam mendeteksi siklus karena memiliki kompleksitas waktu $O(n)$ dan kompleksitas ruang $O(1)$.

5.2. Saran

1. Mahasiswa disarankan untuk memahami konsep pointer dan referensi sebelum mempelajari Linked List lebih lanjut.
2. Perlu dilakukan latihan implementasi berbagai jenis Linked List seperti Doubly Linked List dan Circular Linked List.
3. Praktikum selanjutnya dapat dikembangkan dengan penerapan Linked List pada kasus nyata seperti manajemen data dan algoritma pencarian.
4. Pengujian program sebaiknya dilakukan dengan berbagai variasi data untuk memastikan keandalan implementasi.

DAFTAR PUSTAKA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data structures and algorithms in Python. John Wiley & Sons.
- Miller, B. N., & Ranum, D. L. (2011). Problem solving with algorithms and data structures using Python. Franklin, Beedle & Associates.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
- Sharma, D. (2021). Data structures and algorithms in Python. BPB Publications.
- Zulman, M. R. (2026). *Praktikum Algoritma dan Struktur Data: JavaScript & Node.js – Modul 6: Struktur Data Linked List* [Modul praktikum].

LAPORAN PRAKTIKUM

Algoritma & Struktur Data

Stack & Queue Lanjutan



DISUSUN OLEH:

FAQROEL RAZI

2025573010033

TEKNIK INFORMATIKA

POLITEKNIK NEGERI LHOKSEUMAWE

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum yang berjudul "**Stack dan Queue Lanjutan**" dengan baik dan tepat waktu. Laporan ini disusun sebagai bentuk dokumentasi hasil praktikum sekaligus sebagai pemenuhan tugas pada mata kuliah Struktur Data.

Praktikum ini membahas implementasi struktur data Stack dan Queue menggunakan pendekatan Linked List. Melalui kegiatan praktikum, mahasiswa dapat memahami konsep penyimpanan data secara dinamis, mekanisme operasi Stack dan Queue, serta penerapan operasi enqueue dan dequeue menggunakan struktur data Linked List. Pemahaman tersebut menjadi dasar penting dalam pengembangan berbagai aplikasi dan sistem perangkat lunak.

Penulis mengucapkan terima kasih kepada **Bapak M. Reza Zulman, M.Sc.** selaku dosen pengampu yang telah memberikan arahan, bimbingan, dan materi selama pelaksanaan praktikum. Ucapan terima kasih juga disampaikan kepada seluruh pihak yang telah membantu dalam penyusunan laporan ini.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk penyempurnaan laporan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai implementasi struktur data Stack dan Queue.

Lhokseumawe, 5 Juni 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM.....	v
KATA PENGANTAR.....	vi
DAFTAR ISI.....	vii
BAB I PENDAHULUAN.....	20
1.1 Latar Belakang	20
1.2 Identifikasi Masalah	20
1.3 Rumusan Masalah	21
BAB II LANDASAN TEORI	22
2.1 Pengertian Stack.....	22
2.2 Mengapa Stack Menggunakan Linked List	22
2.3 Pengertian Queue	23
2.4 Mengapa Queue Menggunakan Linked List	23
2.5 Enqueue dan Dequeue.....	24
BAB III METODOLOGI PRAKTIKUM.....	25
3.1 Alat dan Bahan	25
3.2 Prosedur Praktikum.....	25
3.2.1 Setup project.....	25
3.2.2 Melakukan tugas-1 Simulasi Sistem Antrian RS	26
3.2.3 Melakukan tugas-2 Min Stack	30
3.2.4 Melakukan integrasi data antara komputer lokal dengan repositori GitHub.....	33
BAB IV HASIL DAN PEMBAHASAN.....	34
4.1 Hasil Praktikum.....	34
4.2 Pembahasan.....	34
BAB V PENUTUP	35
5.1. Kesimpulan	35
5.2. Saran.....	35

DAFTAR PUSTAKA.....	36
----------------------------	-----------

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan komponen penting dalam ilmu komputer yang digunakan untuk mengatur dan mengelola data secara efisien. Pemilihan struktur data yang tepat dapat meningkatkan performa program serta mempermudah proses pengolahan informasi. Oleh karena itu, pemahaman terhadap berbagai jenis struktur data menjadi hal yang penting bagi mahasiswa teknologi informasi.

Dua struktur data yang sering digunakan dalam pengembangan perangkat lunak adalah Stack dan Queue. Keduanya memiliki karakteristik dan mekanisme kerja yang berbeda, di mana Stack menggunakan prinsip Last In First Out (LIFO) sedangkan Queue menggunakan prinsip First In First Out (FIFO). Struktur data tersebut banyak diterapkan dalam berbagai sistem komputasi modern.

Pada praktikum ini dilakukan implementasi Stack dan Queue menggunakan struktur data Linked List. Pendekatan ini memungkinkan pengelolaan data secara dinamis tanpa keterbatasan ukuran tetap seperti pada array. Selain itu, mahasiswa mempelajari operasi-operasi utama seperti push, pop, enqueue, dan dequeue untuk memahami cara kerja kedua struktur data tersebut secara lebih mendalam.

1.2 Identifikasi Masalah

Berdasarkan latar belakang tersebut, dapat diidentifikasi beberapa permasalahan sebagai berikut:

1. Bagaimana konsep dasar struktur data Stack.
2. Bagaimana konsep dasar struktur data Queue.
3. Mengapa Linked List digunakan dalam implementasi Stack dan Queue.
4. Bagaimana proses enqueue dan dequeue pada Queue.
5. Bagaimana implementasi Stack dan Queue menggunakan Linked List.

1.3 Rumusan Masalah

Adapun rumusan masalah pada praktikum ini adalah sebagai berikut:

1. Apa yang dimaksud dengan Stack dan bagaimana prinsip kerjanya?
2. Apa yang dimaksud dengan Queue dan bagaimana prinsip kerjanya?
3. Mengapa Linked List cocok digunakan untuk implementasi Stack dan Queue?
4. Bagaimana mekanisme enqueue dan dequeue pada Queue?
5. Bagaimana implementasi operasi dasar Stack dan Queue menggunakan Linked List?

BAB II

LANDASAN TEORI

2.1 Pengertian Stack

Stack adalah struktur data linear yang bekerja berdasarkan prinsip **Last In First Out (LIFO)**. Prinsip ini berarti elemen terakhir yang masuk ke dalam Stack akan menjadi elemen pertama yang keluar.

Operasi utama pada Stack meliputi:

- Push : Menambahkan data ke puncak Stack.
- Pop : Menghapus data dari puncak Stack.
- Peek/Top : Melihat data paling atas tanpa menghapusnya.
- IsEmpty : Memeriksa apakah Stack kosong.

Contoh penerapan Stack dapat ditemukan pada fitur Undo-Redo, riwayat browser, dan pemanggilan fungsi pada program.

2.2 Mengapa Stack Menggunakan Linked List

Implementasi Stack menggunakan Linked List memberikan fleksibilitas dalam pengelolaan memori karena ukuran Stack dapat bertambah atau berkurang secara dinamis sesuai kebutuhan. Berbeda dengan array yang memiliki ukuran tetap, Linked List memungkinkan penambahan node baru tanpa perlu menentukan kapasitas awal.

Keuntungan penggunaan Linked List pada Stack:

- Tidak memiliki batas ukuran tetap.
- Operasi push dan pop dapat dilakukan dengan efisien.
- Penggunaan memori lebih fleksibel.
- Tidak memerlukan proses pergeseran data.

2.3 Pengertian Queue

Queue adalah struktur data linear yang bekerja berdasarkan prinsip **First In First Out (FIFO)**. Prinsip ini berarti data yang pertama masuk akan menjadi data pertama yang keluar.

Operasi utama Queue meliputi:

- Enqueue : Menambahkan data ke bagian belakang Queue.
- Dequeue : Menghapus data dari bagian depan Queue.
- Front : Melihat elemen terdepan.
- Rear : Melihat elemen terakhir.
- IsEmpty : Memeriksa apakah Queue kosong.

Contoh penggunaan Queue dapat ditemukan pada sistem antrean pelanggan, penjadwalan proses sistem operasi, dan pengelolaan antrian printer.

2.4 Mengapa Queue Menggunakan Linked List

Linked List sangat cocok digunakan untuk Queue karena memungkinkan penambahan dan penghapusan data secara efisien pada bagian depan maupun belakang struktur data. Dengan adanya pointer head dan tail, operasi enqueue dan dequeue dapat dilakukan tanpa perlu menggeser elemen lain.

Keuntungan Queue berbasis Linked List:

- Kapasitas bersifat dinamis.
- Operasi enqueue dan dequeue lebih efisien.
- Tidak memerlukan alokasi memori berukuran tetap.
- Cocok untuk data yang terus berubah.

2.5 Enqueue dan Dequeue

Enqueue

Enqueue adalah proses menambahkan elemen baru ke bagian belakang Queue (rear). Ketika data baru ditambahkan, pointer tail akan diperbarui sehingga menunjuk ke node terbaru.

Contoh:

Sebelum:

10 → 20 → 30

Enqueue(40)

Sesudah:

10 → 20 → 30 → 40

Dequeue

Dequeue adalah proses menghapus elemen yang berada di bagian depan Queue (front). Setelah data dihapus, pointer head akan berpindah ke node berikutnya.

Contoh:

Sebelum:

10 → 20 → 30 → 40

Dequeue()

Sesudah:

20 → 30 → 40

Operasi enqueue dan dequeue merupakan inti dari mekanisme FIFO yang digunakan pada Queue.

BAB III

METODOLOGI PRAKTIKUM

3.1 Alat dan Bahan

Berdasarkan praktikum yang dilakukan, alat dan bahan yang diperlukan adalah:

- Komputer atau Laptop
- IDE Visual Studio Code
- Node JS
- Git
- GitHub

3.2 Prosedur Praktikum

3.2.1 Setup project

Membuat folder tugas untuk menyimpan project website todo list:

8. Buka git bash dan navigasikan ke folder repository GitHub yang telah di clone.
9. Buat direktori praktikum-7
10. Masuk ke direktori praktikum-7 lalu
11. Buat project node didalam direktori
12. Buat direktori tugas
13. Masuk ke direktori tugas lalu
14. Buat file tugas-1.js dan tugas-2.js

```

ASUSQ LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma (main)
• $ mkdir praktikum_7

ASUSQ LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma (main)
• $ cd praktikum_7/

ASUSQ LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_7 (main)
• $ npm init -y
Wrote to D:\2025573010033_Prak-Struktur-Data-dan-Algoritma\praktikum_7\package.json:

{
  "name": "praktikum_7",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

ASUSQ LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_7 (main)
• $ mkdir tugas

ASUSQ LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_7/tugas (main)
• $ cd tugas

ASUSQ LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_7/tugas (main)
• $ touch tugas-1.js tugas-2.js

ASUSQ LAPTOP-F10SDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma/praktikum_7/tugas (main)
○ $

```

3.2.2 Melakukan tugas-1 Simulasi Sistem Antrian RS

Buka file tugas-1.js yang telah dibuat dan buatlah code javascript untuk memenuhi permintaan tugas pada modul praktikum:

Source code tugas-1.js:

```
tugas-1.js X
praktikum-7 > tugas > tugas-1.js > ...
Windsurf: Refactor | Explain
1 class Node {
  Windsurf: Refactor | Explain | Generate JSDoc
2   constructor(data) {
3     this.data = data;
4     this.next = null;
5   }
6 }
7
Windsurf: Refactor | Explain
8 class Queue {
  Windsurf: Refactor | Explain | Generate JSDoc
9   constructor() {
10    this.head = null;
11    this.tail = null;
12    this.size = 0;
13  }
14
  Windsurf: Refactor | Explain | Generate JSDoc
15  enqueue(data) {
16    const node = new Node(data);
17    if (!this.tail) {
18      this.head = this.tail = node;
19    } else {
20      this.tail.next = node;
21      this.tail = node;
22    }
23    this.size++;
24  }
25
  Windsurf: Refactor | Explain | Generate JSDoc
26  dequeue() {
27    if (this.isEmpty()) return null;
28    const val = this.head.data;
29    this.head = this.head.next;
30    if (!this.head) this.tail = null;
31    this.size--;
32    return val;
33  }
34
```

```

tugas-1.js x
praktikum-7 > tugas > tugas-1.js > ...
35  ...front() {
36  ...    return this.head ? this.head.data : null;
37  ...}
38
    Windsurf: Refactor | Explain | Generate JSDoc
39  ...isEmpty() {
40  ...    return this.size === 0;
41  ...}
42
    Windsurf: Refactor | Explain | Generate JSDoc
43  ...toArray() {
44  ...    const arr = [];
45  ...    let cur = this.head;
46  ...    while (cur) {
47  ...        arr.push(cur.data);
48  ...        cur = cur.next;
49  ...    }
50  ...    return arr;
51  ...}
52  }
53
    Windsurf: Refactor | Explain
54  class Pasien {
    Windsurf: Refactor | Explain | Generate JSDoc
55  ...constructor(id, nama, prioritas) {
56  ...    this.id = id;
57  ...    this.nama = nama;
58  ...    this.prioritas = prioritas; // 'darurat' or 'biasa'
59  ...    this.waktuDaftar = new Date();
60  ...}
61  }
62
    Windsurf: Refactor | Explain
63  class AntrianRS {
    Windsurf: Refactor | Explain | Generate JSDoc
64  ...constructor() {
65  ...    this.antrianDarurat = new Queue();
66  ...    this.antrianBiasa = new Queue();
67  ...}
68

```

```

tugas-1.js x
praktikum-7 > tugas > tugas-1.js > ...
69  ...daftar(pasien) {
70  ...    if (pasien.prioritas === "darurat") {
71  ...        this.antrianDarurat.enqueue(pasien);
72  ...        console.log('Daftar: [DARURAT] ${pasien.nama} (id:${pasien.id})');
73  ...    } else {
74  ...        this.antrianBiasa.enqueue(pasien);
75  ...        console.log('Daftar: [Biasa] ${pasien.nama} (id:${pasien.id})');
76  ...    }
77  ...}
78
    Windsurf: Refactor | Explain | Generate JSDoc
79  ...layani() {
80  ...    let pasien = null;
81  ...    if (!this.antrianDarurat.isEmpty()) {
82  ...        pasien = this.antrianDarurat.dequeue();
83  ...        console.log('Layani (DARURAT): ${pasien.nama} (id:${pasien.id})');
84  ...    } else if (!this.antrianBiasa.isEmpty()) {
85  ...        pasien = this.antrianBiasa.dequeue();
86  ...        console.log('Layani (Biasa): ${pasien.nama} (id:${pasien.id})');
87  ...    } else {
88  ...        console.log("Tidak ada pasien dalam antrian.");
89  ...    }
90  ...    return pasien;
91  ...}
92

```

```

tugas-1.js x
praktikum-7 > tugas > tugas-1.js > ...
93     tampilkanAntrian() {
94         const darurat = this.antrianDarurat
95         .toArray()
96         .map((p) => `${p.nama}(id:${p.id})`);
97         const biasa = this.antrianBiasa
98         .toArray()
99         .map((p) => `${p.nama}(id:${p.id})`);
100         console.log("\nStatus Antrian:");
101         console.log(
102             "Antrian Darurat ->",
103             darurat.length ? darurat.join(" <- ") : "Kosong",
104         );
105         console.log(
106             "Antrian Biasa ->",
107             biasa.length ? biasa.join(" <- ") : "Kosong",
108         );
109     }
110 }
111
112 // Simulasi: daftarkan 10 pasien secara acak (mix darurat dan biasa)
113 function randomPrioritas() {
114     // probability ~30% darurat, 70% biasa
115     return Math.random() < 0.3 ? "darurat" : "biasa";
116 }
117

```

```

tugas-1.js x
praktikum-7 > tugas > tugas-1.js > ...
118 const namaContoh = [
119     "Andi",
120     "Budi",
121     "Citra",
122     "Dewi",
123     "Eka",
124     "Fajar",
125     "Gina",
126     "Hasan",
127     "Indra",
128     "Joko",
129     "Kiki",
130     "Lina",
131 ];
132
133 const rumahSakit = new AntrianRS();
134
135 // daftarkan 10 pasien
136 for (let i = 1; i <= 10; i++) {
137     const nama = namaContoh[Math.floor(Math.random() * namaContoh.length)];
138     const prioritas = randomPrioritas();
139     const p = new Pasien(i, nama, prioritas);
140     rumahSakit.daftar(p);
141 }
142
143 rumahSakit.tampilkanAntrian();
144
145 console.log("\n--- Mulai Melayani ---");
146 while (
147     !rumahSakit.antrianDarurat.isEmpty() ||
148     !rumahSakit.antrianBiasa.isEmpty()
149 ) {
150     rumahSakit.layani();
151 }
152 console.log("Selesai melayani semua pasien.");
153

```

Hasil yang didapatkan:

```
ASUSLAPTOP-F105G73 MINGWA /d/202597301003_PrakStruktur-Dan-Algoritma/praktikum-7/tugas (main)
• 3 node tugas-1.js
DefVar: [Bianca] Budi (id:1)
DefVar: [Bianca] Kiki (id:2)
DefVar: [Bianca] Hasan (id:3)
DefVar: [Bianca] Andi (id:4)
DefVar: [Bianca] Hasan (id:5)
DefVar: [Bianca] Kiki (id:6)
DefVar: [Bianca] Andi (id:7)
DefVar: [Bianca] Gino (id:8)
DefVar: [Bianca] Dora (id:9)
DefVar: [Bianca] Fajar (id:10)

Status Antrian:
Antrian Berurut -> Kosong
Antrian Biasa -> Budi(id:1) <- Kiki(id:2) <- Hasan(id:3) <- Andi(id:4) <- Hasan(id:5) <- Kiki(id:6) <- Andi(id:7) <- Gino(id:8) <- Dora(id:9) <- Fajar(id:10)

--- Mulai Melayari ---
Layari (Bianca): Budi (id:1)
Layari (Bianca): Kiki (id:2)
Layari (Bianca): Hasan (id:3)
Layari (Bianca): Andi (id:4)
Layari (Bianca): Hasan (id:5)
Layari (Bianca): Kiki (id:6)
Layari (Bianca): Andi (id:7)
Layari (Bianca): Gino (id:8)
Layari (Bianca): Dora (id:9)
Layari (Bianca): Fajar (id:10)
Selesai melayari semua pasien.
```

3.2.3 Melakukan tugas-2 Min Stack

Buka file tugas-2.js yang telah dibuat dan buatlah code javascript untuk memenuhi permintaan tugas pada modul praktikum:

Source code:

```
tugas-2.js X
praktikum-7 > tugas > tugas-2.js > Stack
Windsurf: Refactor | Explain
1 class Node {
  Windsurf: Refactor | Explain | Generate JSDoc
2   constructor(data) {
3     this.data = data;
4     this.next = null;
5   }
6 }
7
8 class Stack {
  Windsurf: Refactor | Explain | Generate JSDoc
9   constructor() {
10    this.top = null;
11    this.size = 0;
12  }
13
14   push(data) {
15     const newNode = new Node(data);
16     newNode.next = this.top;
17     this.top = newNode;
18     this.size++;
19   }
20
21   pop() {
22     if (this.isEmpty()) return null;
23     const val = this.top.data;
24     this.top = this.top.next;
25     this.size--;
26     return val;
27   }
28
29   peek() {
30     return this.top ? this.top.data : null;
31   }
32 }
```

```

tugas-2.js x
praktikum-7 > tugas > tugas-2.js > Stack
33  isEmpty() {
34  ...return this.size === 0;
35  ...}
36  }
37
Windsurf: Refactor | Explain
38  class MinStack {
Windsurf: Refactor | Explain | Generate JSDoc
39  ...constructor() {
40  ...this.stack = new Stack();
41  ...this.minStack = new Stack();
42  ...}
43
Windsurf: Refactor | Explain | Generate JSDoc
44  ...push(value) {
45  ...this.stack.push(value);
46  ...if (this.minStack.isEmpty() || value <= this.minStack.peak()) {
47  ...this.minStack.push(value);
48  ...}
49  ...}
50
Windsurf: Refactor | Explain | Generate JSDoc
51  ...pop() {
52  ...if (this.stack.isEmpty()) return null;
53  ...const val = this.stack.pop();
54  ...if (val === this.minStack.peak()) {
55  ...this.minStack.pop();
56  ...}
57  ...return val;
58  ...}
59
Windsurf: Refactor | Explain | Generate JSDoc
60  ...getMin() {
61  ...return this.minStack.isEmpty() ? null : this.minStack.peak();
62  ...}
63
Windsurf: Refactor | Explain | Generate JSDoc

```

```

tugas-2.js x
praktikum-7 > tugas > tugas-2.js > Stack
64  isEmpty() {
65  ...return this.stack.isEmpty();
66  ...}
67
Windsurf: Refactor | Explain | Generate JSDoc
68  ...print() {
69  ...let s = "TOP -> ";
70  ...cur = this.stack.top;
71  ...while (cur) {
72  ...s += `${cur.data} `;
73  ...cur = cur.next;
74  ...}
75  ...console.log("", s.trim());
76  ...}
77  }
78
79  // --- Uji coba kasus ---
80  console.log("=== Uji MinStack ===\n");
81  const ms = new MinStack();
82
83  console.log("push(5) -> getMin()=5");
84  ms.push(5);
85  console.log("Stack:", ms.stack.top?.data);
86  console.log("Min:  ${ms.getMin()}");
87
88  console.log("\npush(3) -> getMin()=3");
89  ms.push(3);
90  console.log("Stack:", ms.stack.top?.data);
91  console.log("Min:  ${ms.getMin()}");
92
93  console.log("\npush(7) -> getMin()=3");
94  ms.push(7);
95  console.log("Stack:", ms.stack.top?.data);
96  console.log("Min:  ${ms.getMin()}");
97

```

```

tugas-2.js
praktikum-7 > tugas > tugas-2.js > Stack
98 console.log("\npush(2) -> getMin()=2");
99 ms.push(2);
100 console.log(`Stack:`, ms.stack.top7.data);
101 console.log(`Min: ...${ms.getMin()}`);
102
103 console.log("\npop() -> hasilkan 2, getMin()=3");
104 const p1 = ms.pop();
105 console.log(`Pop: ...${p1}`);
106 console.log(`Min: ...${ms.getMin()}`);
107
108 console.log("\npop() -> hasilkan 7, getMin()=3");
109 const p2 = ms.pop();
110 console.log(`Pop: ...${p2}`);
111 console.log(`Min: ...${ms.getMin()}`);
112
113 console.log("\npop() -> hasilkan 3, getMin()=5");
114 const p3 = ms.pop();
115 console.log(`Pop: ...${p3}`);
116 console.log(`Min: ...${ms.getMin()}`);
117
118 console.log("\n=== Big O Analysis ===");
119 console.log("push(value) : O(1) - insert ke dua stack dengan operasi konstan");
120 console.log("pop() : O(1) - hapus dari dua stack dengan operasi konstan",
121 );
122 );
123 console.log("getMin() : O(1) - akses top dari minStack dengan operasi konstan",
124 );
125 );
126 console.log("isEmpty() : O(1) - check size dengan operasi konstan");
127 console.log("print() : O(n) - traverse seluruh stack dari top ke bottom");
128

```

Hasil yang didapatkan:

```

ASUSLAPTOP-F1050GTJ MINGW64 /d/2025573010033_Prak-Struktur-Dan-Algoritma/praktikum-7/tugas (main)
$ node tugas-2.js
=== Uji MinStack ===

push(5) -> getMin()=5
Stack: 5
Min: 5

push(3) -> getMin()=3
Stack: 3
Min: 3

push(7) -> getMin()=3
Stack: 7
Min: 3

push(2) -> getMin()=2
Stack: 2
Min: 2

pop() -> hasilkan 2, getMin()=3
Pop: 2
Min: 3

pop() -> hasilkan 7, getMin()=3
Pop: 7
Min: 3

pop() -> hasilkan 3, getMin()=5
Pop: 3
Min: 5

=== Big O Analysis ===
push(value) : O(1) - insert ke dua stack dengan operasi konstan
pop() : O(1) - hapus dari dua stack dengan operasi konstan
getMin() : O(1) - akses top dari minStack dengan operasi konstan
isEmpty() : O(1) - check size dengan operasi konstan
print() : O(n) - traverse seluruh stack dari top ke bottom

```

3.2.4 Melakukan integrasi data antara komputer lokal dengan repositori GitHub

Lakukan integrasi proyek dari komputer lokal ke repositori GitHub dengan mengunggah seluruh berkas yang diperlukan sehingga setiap perubahan yang dilakukan dapat terdokumentasi dan tersimpan secara aman.

```
ASUSLAPTOP-FLOSDGTJ MINGW64 /d/2025573010033_Prak-Struktur-Deta-dan-Algoritma/praktikum_7 (main)
• $ git add .
warning: in the working copy of 'praktikum_7/package.json', LF will be replaced by CRLF the next time Git touches it

ASUSLAPTOP-FLOSDGTJ MINGW64 /d/2025573010033_Prak-Struktur-Deta-dan-Algoritma/praktikum_7 (main)
• $ git commit -m "Praktikum 7: Stack & Queue Lanjutan"
[main 880b428] Praktikum 7: Stack & Queue Lanjutan
3 files changed, 13 insertions(+)
create mode 100644 praktikum_7/package.json
create mode 100644 praktikum_7/tugas/tugas-1.js
create mode 100644 praktikum_7/tugas/tugas-2.js

ASUSLAPTOP-FLOSDGTJ MINGW64 /d/2025573010033_Prak-Struktur-Deta-dan-Algoritma/praktikum_7 (main)
• $ git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 624 bytes | 312.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Fq1Razi/2025573010033_Prak-Struktur-Deta-dan-Algoritma.git
81a5794..880b428 main -> main

ASUSLAPTOP-FLOSDGTJ MINGW64 /d/2025573010033_Prak-Struktur-Deta-dan-Algoritma/praktikum_7 (main)
○ $
```

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum

Hasil praktikum menunjukkan bahwa konsep Stack dan Queue dapat diimplementasikan dengan baik menggunakan struktur data Linked List. Program yang dikembangkan mampu melakukan pengelolaan data secara dinamis melalui operasi push, pop, enqueue, dan dequeue sesuai dengan prinsip kerja LIFO (Last In First Out) pada Stack dan FIFO (First In First Out) pada Queue. Implementasi ini membuktikan bahwa Linked List dapat digunakan secara efektif untuk mendukung operasi dasar Stack dan Queue dengan penggunaan memori yang lebih fleksibel.

4.2 Pembahasan

Praktikum ini berhasil mengimplementasikan struktur data Stack dan Queue menggunakan Linked List. Implementasi Stack menunjukkan bahwa proses push dan pop dapat dilakukan secara efisien pada bagian atas Stack tanpa memerlukan pergeseran data. Penggunaan Linked List memungkinkan ukuran Stack bertambah maupun berkurang secara dinamis sesuai jumlah data yang disimpan sehingga penggunaan memori menjadi lebih fleksibel dibandingkan implementasi berbasis array.

Selain Stack, praktikum juga berhasil mengimplementasikan Queue menggunakan mekanisme enqueue dan dequeue. Operasi enqueue digunakan untuk menambahkan data ke bagian belakang antrian, sedangkan dequeue digunakan untuk menghapus data dari bagian depan antrian sesuai prinsip FIFO. Hasil pengujian menunjukkan bahwa penggunaan pointer head dan tail pada Linked List mampu meningkatkan efisiensi pengelolaan data serta mempermudah proses manipulasi elemen dalam Queue.

BAB V

PENUTUP

5.1. Kesimpulan

Stack merupakan struktur data yang menggunakan prinsip Last In First Out (LIFO), sedangkan Queue merupakan struktur data yang menggunakan prinsip First In First Out (FIFO). Dalam implementasinya, Linked List memberikan fleksibilitas yang tinggi karena bersifat dinamis dan tidak memiliki batas ukuran tetap seperti array. Pada Stack, operasi utama yang digunakan adalah push dan pop untuk menambahkan serta menghapus data dari puncak Stack. Sementara itu, pada Queue digunakan operasi enqueue dan dequeue untuk menambahkan data ke bagian belakang dan menghapus data dari bagian depan antrean. Dengan menggunakan Linked List, pengelolaan data pada Stack dan Queue dapat dilakukan secara lebih efisien dibandingkan array pada kondisi tertentu, terutama ketika ukuran data sering berubah selama program berjalan.

5.2. Saran

1. Mahasiswa disarankan memahami konsep Linked List sebelum mengimplementasikan Stack dan Queue.
2. Praktikum dapat dikembangkan dengan membuat aplikasi sederhana yang memanfaatkan Stack atau Queue.
3. Pengujian program sebaiknya dilakukan dengan jumlah data yang bervariasi.
4. Perlu dilakukan eksplorasi implementasi struktur data lain seperti Tree dan Graph untuk memperluas pemahaman.

DAFTAR PUSTAKA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Python*. John Wiley & Sons.
- Miller, B. N., & Ranum, D. L. (2011). *Problem solving with algorithms and data structures using Python*. Franklin, Beedle & Associates.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
- Sharma, D. (2021). *Data structures and algorithms in Python*. BPB Publications.
- Zulman, M. R. (2026). *Praktikum Algoritma dan Struktur Data: JavaScript & Node.js – Modul 7 - Stack & Queue Lanjutan* [Modul praktikum].